

MINISTRY OF EDUCATION OF REPUBLIC OF MOLDOVA
TECHNICAL UNIVERSITY OF MOLDOVA
FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
SOFTWARE ENGINEERING DEPARTMENT

EMBEDDED SYSTEMS
LABORATORY WORK WORK #6.2

Button-Controlled LED FSM with Debouncing

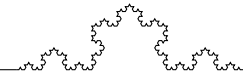
During the preparation of this report, the author used various AI Tools such as ChatGPT, Gemini, Claude to generate/augment the content, generate the code and structure the directory. The resulting information was reviewed, validated, and adjusted to meet the requirements of the laboratory assignment.

Author: Timur CRAVTOV
std. gr. FAF-231

Verified by:
Alexei MARTÎNIUC
asist. univ



Chişinău, 2026



Contents

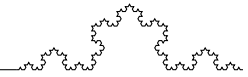
1	Objective and Technical Analysis	1
1.1	Specifications	2
1.2	Finite State Machine	2
1.3	Debouncing and Rate Limiting	2
2	System Design	3
2.1	Hardware Wiring	3
2.2	Software Architecture	3
2.3	Program Flow	4
2.4	Layered Hardware/Software Design	6
2.4.1	LED Layered Design	6
3	Implementation and Results	7
4	Conclusions	7
	References	7
5	Appendix - Source Code	9
5.1	main.cpp	9
5.2	button_led_controller.cpp	10
5.3	led_fsm.cpp	11
5.4	debouncer.cpp	12

1 Objective and Technical Analysis

The objective of this laboratory work is to implement a finite state machine (FSM) that controls an LED based on a debounced push-button input. The system has two states (OFF and ON) and transitions at each validated button press. A lockout window prevents rapid repeat toggles, and the current LED state is reported in real time over the serial interface. The solution is modularized into small components (debouncer, FSM, controller), enabling clear separation of responsibilities and easier extension.

The main learning outcomes are:

- modeling binary actuator control with a simple FSM;
- implementing robust software debouncing for mechanical buttons;
- applying rate limiting to avoid repeated toggles under noisy input;
- structuring embedded code into reusable modules (FSM, debounce, controller).



1.1 Specifications

Table 1: Lab 6.2 Specifications

Item	Specification
MCU	Arduino Nano (ATmega328P)
Input	Push-button (D5) with pull-down resistor
Output	LED (D2) with $220\ \Omega$ resistor
FSM states	OFF, ON
Debounce time	40 ms
Toggle lockout	250 ms
Reporting	Serial (printf) at 9600 baud

1.2 Finite State Machine

The control logic is modeled as a two-state FSM [3, 4, 5]. Each debounced button press generates a `ButtonPressed` event that toggles the state. The transition table is defined explicitly, making it easy to extend the system with additional states or events. State actions are limited to LED output updates and serial reporting, keeping transitions deterministic and easy to verify.

Figure 1 presents the two stable states and the single event that triggers transitions in both directions.

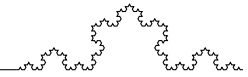
LED Finite State Machine



Figure 1: LED FSM: toggling between OFF and ON on each valid press

1.3 Debouncing and Rate Limiting

Mechanical buttons produce rapid oscillations at the contacts when pressed or released [2]. A time-based debouncer validates a state change only if the raw input remains stable for at least 40 ms. After a valid press, a 250 ms lockout interval prevents repeated toggles from accidental rapid clicks or residual bounce. Only the rising edge (stable HIGH) is accepted as a press event, so releases do not affect the FSM.



The debounce logic can be summarized as:

$$\text{stableChange} = (t - t_{\text{lastChange}}) \geq T_{\text{debounce}} \quad (1)$$

and the lockout check as:

$$\text{allowToggle} = (t - t_{\text{lastToggle}}) \geq T_{\text{lockout}} \quad (2)$$

2 System Design

2.1 Hardware Wiring

The wiring is intentionally minimal:

- LED connected to D2 through a 220Ω resistor to ground.
- Push-button wired to D5 with a $10\text{k}\Omega$ pull-down resistor; the other terminal goes to 5V.

This arrangement ensures that the input reads LOW by default and HIGH when the button is pressed. The LED resistor limits the current to a safe value, and the pull-down resistor guarantees a defined logic level when the button is released. Figure 2 summarizes the electrical connections between the Nano, the LED, and the button.

Electrical Scheme

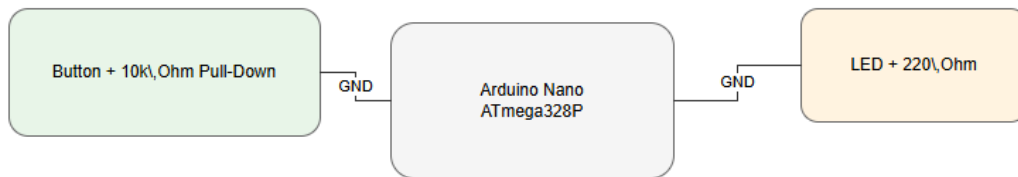


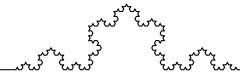
Figure 2: Electrical scheme for the button and LED connections

2.2 Software Architecture

The implementation is decomposed into three small modules:

- **Debouncer:** time-based stabilization for raw button input.
- **LED FSM:** table-driven transitions between OFF and ON.
- **Controller:** integrates debouncing, lockout timing, and FSM dispatch.

The main loop simply samples the raw input and forwards it to the controller, keeping `main.cpp` minimal and declarative.



2.3 Program Flow

The flow diagram in Figure 3 shows the runtime sequence: raw input sampling, de-bounce stabilization, press qualification, lockout validation, FSM dispatch, and hardware update. Each branch returns quickly to the loop, minimizing latency and keeping the system responsive. The decision nodes match the controller's guard conditions.

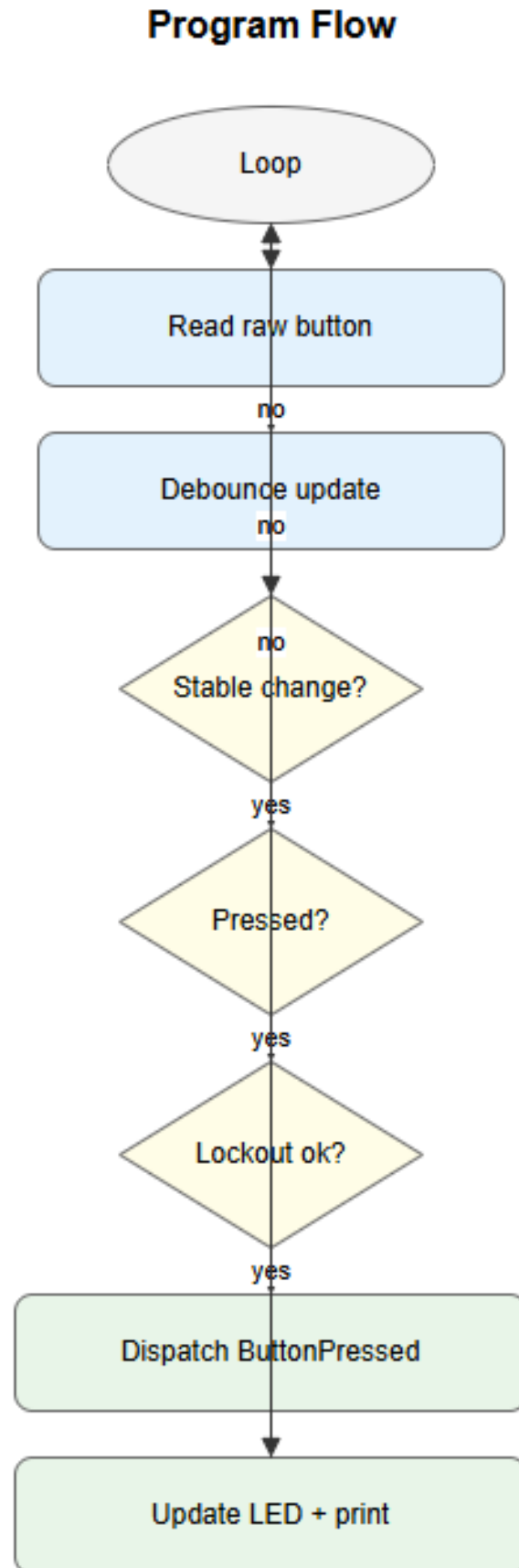
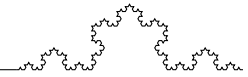
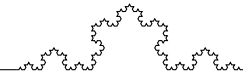


Figure 3: Program flow for debouncing, lockout, and FSM dispatch



2.4 Layered Hardware/Software Design

The layered design clarifies dependencies and isolates hardware access from application logic. The controller coordinates input conditioning and state updates, while drivers abstract pin-level operations. This structure simplifies testing and allows the FSM or debounce strategy to evolve independently. Figure 4 presents the overall layering of the software modules.

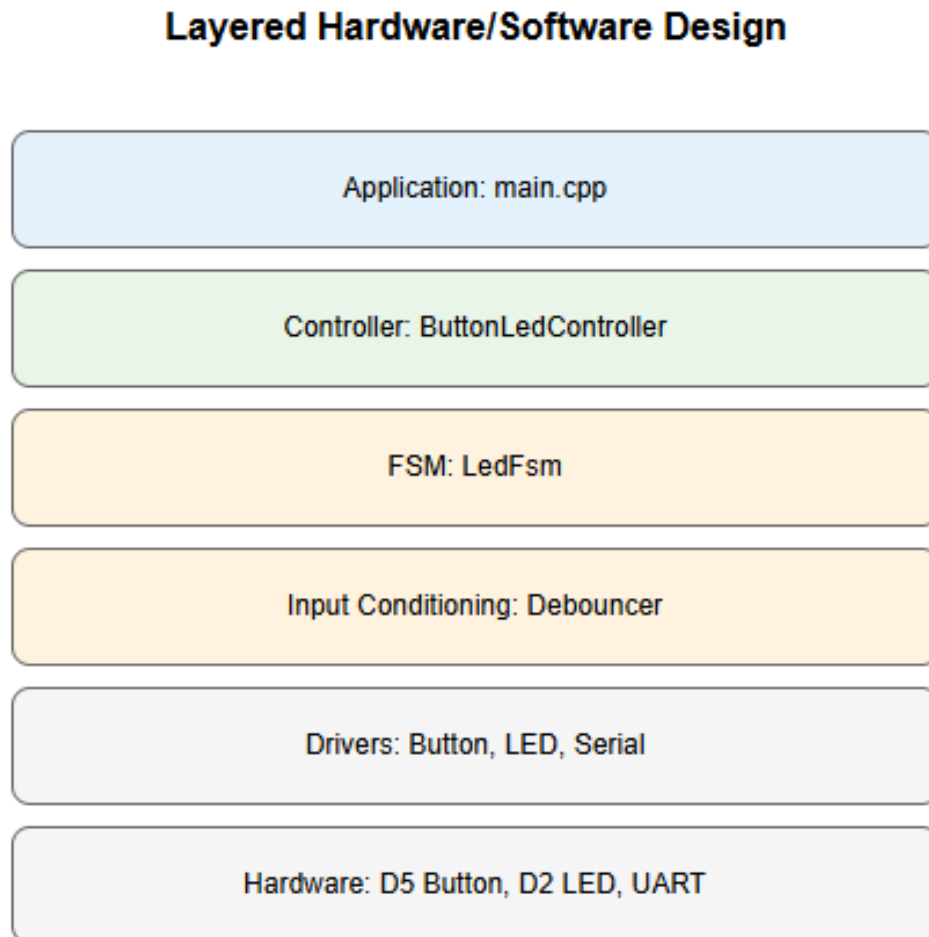


Figure 4: Layered hardware/software architecture

2.4.1 LED Layered Design

The LED control follows a classic APP–SRV–ECAL–MCAL–MCU–ECU stack. The application layer requests state changes, service logic maps states to LED commands, and the lower layers translate those commands into GPIO writes. Figure 5 keeps the LED stack consistent with the layered style used in earlier labs.

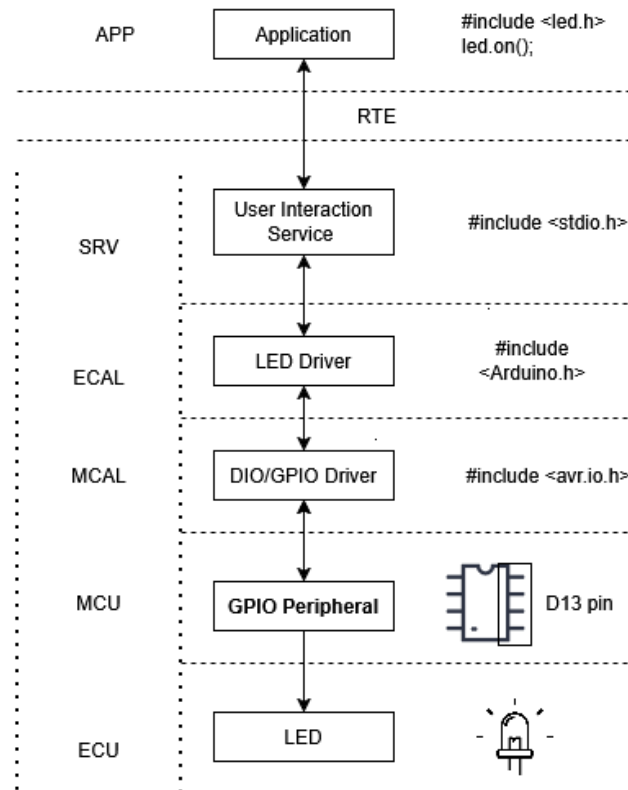
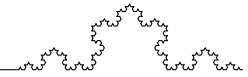


Figure 5: LED layered design (hardware/software stack)

3 Implementation and Results

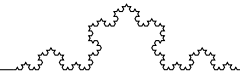
The FSM is implemented as a table of transitions (state + event $\rightarrow$ next state). The controller emits a **ButtonPressed** event on a validated rising edge and updates the LED hardware and serial output. In Wokwi simulation, repeated rapid clicks are ignored until the lockout interval elapses, while single presses toggle the LED deterministically. The debounce interval and lockout duration can be tuned to balance responsiveness and noise immunity.

4 Conclusions

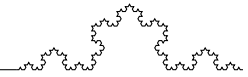
This lab demonstrates a practical FSM-based design for a binary actuator. By combining debouncing and rate limiting, the system avoids spurious toggles and behaves predictably. The modular structure (debouncer, FSM, controller) improves readability and can be reused for more complex state machines with additional events and outputs.

References

- [1] Arduino Nano Documentation, <https://docs.arduino.cc/hardware/nano>



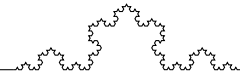
- [2] Jack G. Ganssle, *A Guide to Debouncing*, <https://www.ganssle.com/debouncing.htm>
- [3] Finite-state machine, https://en.wikipedia.org/wiki/Finite-state_machine
- [4] Moore and Mealy Machines, https://en.wikipedia.org/wiki/Moore_machine
- [5] Nandland State Machines Tutorial, <https://nandland.com/state-machines/>
- [6] Wokwi Simulator, <https://wokwi.com/>



5 Appendix - Source Code

5.1 main.cpp

```
1 #include <Arduino.h>
2 #include <button/button.h>
3 #include <led/led.h>
4 #include <serialio/serialio.h>
5
6 #include "controller/button_led_controller.h"
7
8 constexpr uint8_t LED_PIN = 2;
9 constexpr uint8_t BUTTON_PIN = 5;
10 constexpr uint16_t DEBOUNCE_MS = 40;
11 constexpr uint16_t MIN_TOGGLE_INTERVAL_MS = 250;
12
13 Led led(LED_PIN);
14 Button button(BUTTON_PIN);
15
16 static ButtonLedController controller(&led, DEBOUNCE_MS,
17                                       MIN_TOGGLE_INTERVAL_MS);
18
19 void setup() {
20     led.init();
21     button.init();
22
23     Serial.begin(9600);
24     redirectSerialToStdio(true, true, true);
25
26     const bool initialRaw = button.isPressed();
27     controller.begin(initialRaw, millis());
28 }
29
30 void loop() {
31     const uint32_t now = millis();
32     const bool rawPressed = button.isPressed();
33
34     controller.update(rawPressed, now);
```

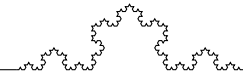


```
35     delay(5);
36 }
```

Listing 1: Main Setup and Loop

5.2 button_led_controller.cpp

```
1 #include "controller/button_led_controller.h"
2
3 #include <stdio.h>
4
5 ButtonLedController::ButtonLedController(Led* led, uint16_t
    debounceMs, uint16_t minToggleIntervalMs)
6     : debouncer_(debounceMs),
7       fsm_(led),
8       lastToggleMs_(0),
9       minToggleIntervalMs_(minToggleIntervalMs) {}
10
11 void ButtonLedController::begin(bool initialRaw, uint32_t nowMs)
12 {
13     debouncer_.begin(initialRaw, nowMs);
14     fsm_.begin();
15     printf("LED: %s\n", ledStateToString(fsm_.state()));
16 }
17
18 void ButtonLedController::update(bool raw, uint32_t nowMs) {
19     if (!debouncer_.update(raw, nowMs)) {
20         return;
21     }
22
23     if (!debouncer_.stable()) {
24         return;
25     }
26
27     if ((uint32_t)(nowMs - lastToggleMs_) < minToggleIntervalMs_) {
28         return;
29     }
```



```

30     if (fsm_.dispatch(Event::ButtonPressed)) {
31         printf("LED: %s\n", ledStateToString(fsm_.state()));
32         lastToggleMs_ = nowMs;
33     }
34 }

```

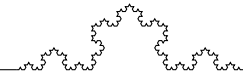
Listing 2: Controller Integration

5.3 led_fsm.cpp

```

1 #include "fsm/led_fsm.h"
2
3 struct Transition {
4     LedState from;
5     Event event;
6     LedState to;
7 };
8
9 static const Transition FSM_TRANSITIONS[] = {
10     {LedState::Off, Event::ButtonPressed, LedState::On},
11     {LedState::On, Event::ButtonPressed, LedState::Off}
12 };
13
14 LedFsm::LedFsm(Led* led) : led_(led), state_(LedState::Off) {}
15
16 void LedFsm::begin() {
17     apply(state_);
18 }
19
20 bool LedFsm::dispatch(Event event) {
21     const LedState prev = state_;
22
23     for (const Transition& t : FSM_TRANSITIONS) {
24         if (t.from == state_ && t.event == event) {
25             apply(t.to);
26             break;
27         }
28     }
29 }

```

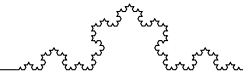


```
30     return state_ != prev;
31 }
32
33 LedState LedFsm::state() const {
34     return state_;
35 }
36
37 void LedFsm::apply(LedState next) {
38     state_ = next;
39
40     if (!led_) {
41         return;
42     }
43
44     if (state_ == LedState::On) {
45         led_ -> on();
46     } else {
47         led_ -> off();
48     }
49 }
50
51 const char* ledStateToString(LedState state) {
52     return (state == LedState::On) ? "ON" : "OFF";
53 }
```

Listing 3: LED FSM Implementation

5.4 debouncer.cpp

```
1 #include "input/debouncer.h"
2
3 Debouncer::Debouncer(uint16_t debounceMs)
4     : stable_(false), lastRaw_(false), lastChangeMs_(0),
5       debounceMs_(debounceMs) {}
6
7 void Debouncer::begin(bool initialRaw, uint32_t nowMs) {
8     stable_ = initialRaw;
9     lastRaw_ = initialRaw;
10    lastChangeMs_ = nowMs;
```



```
10 }
11
12 bool Debouncer::update(bool raw, uint32_t nowMs) {
13     if (raw != lastRaw_) {
14         lastRaw_ = raw;
15         lastChangeMs_ = nowMs;
16     }
17
18     if ((uint32_t)(nowMs - lastChangeMs_) >= debounceMs_ && raw
19         != stable_) {
20         stable_ = raw;
21         return true;
22     }
23
24     return false;
25 }
26
27 bool Debouncer::stable() const {
28     return stable_;
```

Listing 4: Debounce Utility